Sorting

---

20. Selection Sort

Selection sort works by **repeatedly finding the minimum element** from the unsorted part of the array and placing it at the beginning.

At every step, the array is divided into two parts:

- Left part → already sorted
- Right part → unsorted

We select the smallest element from the unsorted part and swap it with the first unsorted position.

Example
Array: `[64, 25, 12, 22, 11]`

Pass 1: minimum = 11 → swap with 64 → `[11, 25, 12, 22, 64]`
Pass 2: minimum = 12 → `[11, 12, 25, 22, 64]`
Pass 3: minimum = 22 → `[11, 12, 22, 25, 64]`

Algorithm

1. Loop from index `0` to `n-2`.
2. Assume the current index is the minimum.
3. Traverse the remaining array to find the actual minimum element.
4. Swap the minimum element with the current index.
5. Repeat until the array is sorted.

Code

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    selectionSort(arr);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n²)
  Two nested loops always run, regardless of input.
- **Space Complexity:** O(1)
  Sorting is done in-place.

---

21. Bubble Sort

Bubble sort repeatedly **compares adjacent elements** and swaps them if they are in the wrong order.
After each pass, the **largest element moves to the end**, like a bubble rising up.

Example
Array: `[5, 1, 4, 2, 8]`

Pass 1: `[1, 4, 2, 5, 8]`
Pass 2: `[1, 2, 4, 5, 8]`
Array becomes sorted.

Algorithm

1. Repeat passes over the array.
2. Compare adjacent elements.
3. Swap if left element is greater than right.

4. After each pass, the largest element settles at the end.
5. Stop early if no swaps occur in a pass.

Code

```
#include <bits/stdc++.h>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

int main() {
    vector<int> arr = {5, 1, 4, 2, 8};
    bubbleSort(arr);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
    - Worst / Average: $O(n^2)$
    - Best (already sorted): $O(n)$
- **Space Complexity:** $O(1)$
  In-place sorting.

---

22. Insertion Sort

Insertion sort builds the sorted array **one element at a time**, similar to how we sort playing cards in our hand.

The current element is inserted into its correct position in the already sorted part.

Example
Array: `[8, 3, 5, 2]`

Insert 3 → `[3, 8, 5, 2]`
Insert 5 → `[3, 5, 8, 2]`
Insert 2 → `[2, 3, 5, 8]`

Algorithm

1. Start from the second element.
2. Compare it with elements before it.
3. Shift larger elements one position to the right.
4. Insert the current element at the correct position.
5. Repeat for all elements.

Code

```
#include <bits/stdc++.h>
using namespace std;

void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    vector<int> arr = {8, 3, 5, 2};
    insertionSort(arr);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
  - Worst / Average: O(n²)
  - Best (already sorted): O(n)
- **Space Complexity:** O(1)
  Sorting is done in-place.

Quick Comparison

Algorithm	Best Case	Average Case	Worst Case	Stable
Selection Sort	O(n²)	O(n²)	O(n²)	No
Bubble Sort	O(n)	O(n²)	O(n²)	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	Yes

These sorting algorithms are mainly used for **learning purposes and small datasets**.

23. Merge Sort

Merge Sort is a **divide and conquer** algorithm.
It divides the array into two halves, sorts each half recursively, and then **merges** the two sorted halves into one sorted array.

Example
Array: `[5, 2, 4, 6, 1, 3]`

Divide → `[5,2,4]` and `[6,1,3]`
Sort left → `[2,4,5]`
Sort right → `[1,3,6]`
Merge → `[1,2,3,4,5,6]`

Algorithm

1. If the array has only one element, it is already sorted.
2. Find the middle of the array.
3. Recursively apply merge sort on left half.
4. Recursively apply merge sort on right half.
5. Merge the two sorted halves into one sorted array.

Code

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    vector<int> left(n1), right(n2);

    for (int i = 0; i < n1; i++)
        left[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        right[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (left[i] <= right[j])
            arr[k++] = left[i++];
        else
            arr[k++] = right[j++];
    }

    while (i < n1)
        arr[k++] = left[i++];

    while (j < n2)
        arr[k++] = right[j++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;

    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
```

```
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}

int main() {
    vector<int> arr = {5, 2, 4, 6, 1, 3};
    mergeSort(arr, 0, arr.size() - 1);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n log n)
  Array is divided into halves and merged at each level.
- **Space Complexity:** O(n)
  Extra space is used for temporary arrays during merging.

---

24. Recursive Bubble Sort

Recursive Bubble Sort performs the same operation as normal bubble sort, but instead of using loops, it uses **recursion**.

In each recursive call, the **largest element moves to the end**, and the problem size reduces by one.

Example
Array: `[4, 3, 2, 1]`

Pass 1 → `[3, 2, 1, 4]`
Pass 2 → `[2, 1, 3, 4]`
Pass 3 → `[1, 2, 3, 4]`

Algorithm

1. If the size of the array is 1, stop.
2. Perform one full pass to bubble the largest element to the end.
3. Recursively call bubble sort for the remaining array of size `n-1`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void recursiveBubbleSort(vector<int>& arr, int n) {
    if (n == 1) return;

    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1])
            swap(arr[i], arr[i + 1]);
    }

    recursiveBubbleSort(arr, n - 1);
}

int main() {
    vector<int> arr = {4, 3, 2, 1};
    recursiveBubbleSort(arr, arr.size());
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n²)
  Same number of comparisons as iterative bubble sort.
- **Space Complexity:** O(n)
  Recursive call stack uses extra space.

---

25. Recursive Insertion Sort

Recursive Insertion Sort sorts the array by **recursively sorting the first n−1 elements** and then inserting the last element into its correct position.

Example
Array: [5, 3, 4, 1]

Sort [5,3,4] → [3,4,5]
Insert 1 → [1,3,4,5]

Algorithm

1. If the size of the array is 1, stop.
2. Recursively sort the first n−1 elements.
3. Insert the last element into the sorted part by shifting elements.

Code

```
#include <bits/stdc++.h>
using namespace std;

void recursiveInsertionSort(vector<int>& arr, int n) {
    if (n <= 1) return;

    recursiveInsertionSort(arr, n - 1);

    int last = arr[n - 1];
    int j = n - 2;

    while (j >= 0 && arr[j] > last) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = last;
}

int main() {
    vector<int> arr = {5, 3, 4, 1};
    recursiveInsertionSort(arr, arr.size());
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
    - Worst / Average: O(n²)
    - Best (already sorted): O(n)
- **Space Complexity:** O(n)
  Due to recursive call stack.

Summary Comparison

Algorithm	Time Complexity	Space Complexity	Stable
Merge Sort	O(n log n)	O(n)	Yes
Recursive Bubble Sort	O(n²)	O(n)	Yes
Recursive Insertion Sort	O(n²) / O(n best)	O(n)	Yes

26. Quick Sort

Quick Sort is a **divide and conquer** sorting algorithm.
It works by selecting a **pivot element**, placing it at its correct position, and then recursively sorting the elements on the left and right of the pivot.

Unlike merge sort, quick sort sorts the array **in-place**.

Question Explanation

You are given an array of integers.
Your task is to sort the array using the **Quick Sort** algorithm.

The idea is:

- Pick one element as a pivot.
- Rearrange the array so that:
    - All elements smaller than the pivot are on the left.
    - All elements greater than the pivot are on the right.
- Recursively apply the same steps to the left and right subarrays.

Approach

Quick Sort (Using Lomuto Partition)

Algorithm

1. Choose the **last element** of the array as the pivot.
2. Rearrange the array such that:
    - Elements smaller than or equal to pivot come before it.
    - Elements greater than pivot come after it.
3. The pivot is now at its correct position.
4. Recursively apply quick sort on:
    - Left subarray (elements before pivot)
    - Right subarray (elements after pivot)
5. Repeat until subarray size becomes 0 or 1.

Example Explanation

Array: `[10, 7, 8, 9, 1, 5]`
Pivot = 5

After partition: `[1, 5, 8, 9, 10, 7]`
Pivot 5 is at correct position.

Left subarray: `[1]`
Right subarray: `[8, 9, 10, 7]`

Apply quick sort recursively to both parts until fully sorted.

Code

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```
void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    quickSort(arr, 0, arr.size() - 1);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
    - Best Case: $O(n \log n)$ (balanced partitions)
    - Average Case: $O(n \log n)$
    - Worst Case: $O(n^2)$ (already sorted array with bad pivot choice)
- Worst case happens because the pivot always ends up at one extreme, leading to highly unbalanced partitions.
- **Space Complexity:** $O(\log n)$
  Due to recursion stack in the average case.

Important Notes

- Quick sort is **not stable**.
- It is usually **faster in practice** than merge sort due to low constant factors.
- Pivot selection plays a very important role in performance.
- Common pivot choices: first element, last element, random element, median of three.